

图论

树的直径
欧拉序
最短路
差分约束
最小生成树
有向图强连通分量
无向图双连通分量
LCA
树上差分
树上启发式合并
树链剖分
拓扑排序
2-SAT
欧拉图
二分图
网络流
最大流之上下界可行流
最大流之技巧
最小割之最大权闭合图、最大密度子图
最小割(二分图)之最小权覆盖集、最大权独立集
最小费用最大流
上下界费用流
Prufer 序列

图论

树的直径

俩遍dfs:

结论: 对于一个树当前的直径端点为 $\langle u, v \rangle$, 当新加一个点 x 到这个树上时, 端点变化为 $\langle u, v \rangle, \langle u, x \rangle, \langle x, v \rangle$ 的其中之一, 即只变一端, 因此可以动态维护

```
1  const int N = 10000 + 10;
2
3  int n, c, d[N];
4  vector<int> E[N];
5
6  void dfs(int u, int fa) {
7      for (int v : E[u]) {
8          if (v == fa) continue;
9          d[v] = d[u] + 1;
10         if (d[v] > d[c]) c = v;
11         dfs(v, u);
12     }
13 }
14
15 int main() {
16     scanf("%d", &n);
17     for (int i = 1; i < n; i++) {
18         int u, v;
19         scanf("%d %d", &u, &v);
20         E[u].push_back(v), E[v].push_back(u);
21     }
22     dfs(1, 0);
23     d[c] = 0, dfs(c, 0);
24     printf("%d\n", d[c]);
25     return 0;
26 }
```

树形dp: 可求负权边

```
1  const int N = 10000 + 10;
2
3  int n, d = 0;
4  int d1[N], d2[N];
```

```

5  vector<int> E[N];
6
7  void dfs(int u, int fa) {
8      d1[u] = d2[u] = 0;
9      for (int v : E[u]) {
10         if (v == fa) continue;
11         dfs(v, u);
12         int t = d1[v] + 1;
13         if (t > d1[u])
14             d2[u] = d1[u], d1[u] = t;
15         else if (t > d2[u])
16             d2[u] = t;
17     }
18     d = max(d, d1[u] + d2[u]);
19 }
20
21 int main() {
22     scanf("%d", &n);
23     for (int i = 1; i < n; i++) {
24         int u, v;
25         scanf("%d %d", &u, &v);
26         E[u].push_back(v), E[v].push_back(u);
27     }
28     dfs(1, 0);
29     printf("%d\n", d);
30     return 0;
31 }

```

欧拉序

查询树上一个点集中的所有路径的中可能的两个端点 u, v : 任意一段序列所表示的边都是其中所有点中的几个点之间的简单路径

- 记录三个值: 点集中最大的 **dfn** 序, 最大最小的 **euler** 序。
- 性质: 最大的 **dfn** 序对应点往下没有别的点, 最小的 **euler** 序往下也没有别的点, 所以这样点能作为一条路径的端点。
- 特殊情况**: 一条链的话, 最大的 **dfn** 序和最下的 **euler** 序是同一个点, 此时用最小的 **euler** 序对应的点即可。

最短路

Floyd算法: 求任意两个节点的最短路, 时间复杂度 $O(n^3)$, 空间复杂度 $O(n^2)$

```

1  auto Floyd = [&](int n) {
2      vector dis(n + 1, vector<int>(n + 1, 4e18));
3      for (int u = 1; u <= n; u++) {
4          dis[u][u] = 0;
5          for (auto & [v, w] : adj[u]) {
6              dis[u][v] = w;
7          }
8      }
9      for (int k = 1; k <= n; k++) {
10         for (int i = 1; i <= n; i++) {
11             for (int j = 1; j <= n; j++) {
12                 dis[i][j] = min(dis[i][j], dis[i][k] + dis[k][j]);
13             }
14         }
15     }
16     return move(dis);
17 };

```

应用: 传递闭包, 判断是否联通, 使用 *Tarjan* 缩点后, 按拓扑序或者记忆化搜索用 *bitset* 存储联通性转移为 $O(n^2/w)$

```

1  bitset<SIZE> f[SIZE];
2  for (int k = 1; k <= n; k++) {
3      for (int i = 1; i <= n; i++) {
4          if (f[i][k]) {
5              f[i] = f[i] | f[k];
6          }
7      }
8  }

```

Bellman-Ford算法: 求单源最短路, 可求负权图, 并判断是否存在负环, 时间复杂度 $O(nm)$

```

1 struct edge {
2     int v, w;
3 };
4 vector<vector<edge>> e(n + 1);
5 vector<int> dis(n + 1, INF);
6
7 bool bellmanford(int n, int s) {
8     dis[s] = 0; //初始化源点
9     bool flag; //判断循环过程是否有松弛操作
10    for (int i = 1; i <= n; i++) { //无负环最多松弛n - 1条
11        flag = 0;
12        for (int u = 1; u <= n; u++) { //对所有点遍历
13            if (dis[u] == inf) {
14                continue;
15            } //无穷大与常数相加减仍为无穷大
16            for (auto [v, w] : e[u]) { //遍历所有边
17                if (dis[v] > dis[u] + w) {
18                    dis[v] = dis[u] + w; //relax操作
19                    flag = 1;
20                }
21            }
22        }
23        if (!flag) {
24            break; //无可继续松弛的边
25        }
26    }
27    return flag; //第n轮仍可以松弛, 有负环
28 }

```

SPFA: 用队列优化的bellman - ford,原理: 被松弛过的边才有可能继续松弛,时间复杂度 $O(nm)$

SLF优化: 将普通队列改为双端队列,每次入队和队首比较,如果更大则插入队尾,否则插入队首

SLF+swap优化: 如果入队的更大就交换队列首尾

```

1 auto SPFA = [&](int s, int n) { // dis[0] < 0 表示有负环
2     vector<int> dis(n + 1, 4e18), vis(n + 1), cnt(n + 1);
3     queue<int> q;
4     dis[s] = 0, vis[s] = 1, q.push(s); // 加入源点
5     while (q.size()) {
6         int u = q.front();
7         q.pop(), vis[u] = 0; // vis 记录 queue 中有什么, 防止重复进队列 mle
8         for (auto & [v, w] : adj[u]) {
9             if (dis[v] > dis[u] + w) {
10                dis[v] = dis[u] + w, cnt[v] = cnt[u] + 1; // 记录最短路经过的边数
11                if (cnt[v] >= n) return dis[0] = -1, move(dis); // 无负环最多经过n - 1条,故有负环
12                if (!vis[v]) {
13                    q.push(v), vis[v] = 1;
14                }
15            }
16        }
17    }
18    return move(dis);
19 };

```

Dijkstra: 计算无负权边图的最短路,时间复杂度 $O(m \log m)$, 若为pbds的pairing_heap则为 $O(m \log n)$

```

1 auto Dijkstra = [&](int s, int n) {
2     vector<int> dis(n + 1, 4e18), vis(n + 1); // vis 记录是否遍历过
3     priority_queue<array<int, 2>, vector<array<int, 2>>, greater<>> q;
4     dis[s] = 0, q.push({dis[s], s});
5     while (q.size()) {
6         int u = q.top()[1];
7         q.pop();
8         if (vis[u]) continue;
9         vis[u] = 1; // 一个点可能在松弛操作多次入堆
10        for (auto & [v, w] : adj[u]) {
11            if (dis[v] > dis[u] + w) {
12                dis[v] = dis[u] + w;
13                q.push({dis[v], v});
14            }
15        }
16    }
17    return move(dis);
18 };
19 auto Dijkstra = [&](int s, int n) { // pbds 的 pairing_heap

```

```

20     vector<int> dis(n + 1, 4e18);
21     pairing_heap<array<int, 2>, greater<>> q;
22     vector<decltype(q)::point_iterator> vis(n + 1); // 记录在堆内位置
23     dis[s] = 0, vis[s] = q.push({dis[s], s});
24     while (q.size()) {
25         int u = q.top()[1];
26         q.pop();
27         for (auto & [v, w] : adj[u]) {
28             if (dis[v] > dis[u] + w) {
29                 dis[v] = dis[u] + w;
30                 if (vis[v] != NULL) {
31                     q.modify(vis[v], {dis[v], v}); // 已经在堆, 则修改
32                 } else {
33                     vis[v] = q.push({dis[v], v});
34                 }
35             }
36         }
37     }
38     return move(dis);
39 };

```

差分约束

作用：对于不等式组, 求出可行解

e.g. $x_v \leq x_u + c$, 可连 $u \rightarrow v$ 权值为 c 的边

然后建立超级源点 s , 求最大可行解时若有 $x_u \leq c$, 建立 $s \rightarrow u$ 权值 c 跑 *SPFA* 最短路

求最小可行解时若有 $x_u \geq c$, 建立 $s \rightarrow u$ 权值 c 跑 *SPFA* 最长路

只求可行解可连 $s \rightarrow u$ 权值 0

最小生成树

定义：边权和最小的生成树, 只有联通图才有生成树

kruskal：对边权进行排序并用并查集检查是否联通, 时间复杂度 $O(m \log m)$

```

1     template <int len = 3> struct Kruskal : vector<array<int, len>> {
2         using e = vector<array<int, len>>; // {w, u, v}
3         DSU dsu;
4         int sum = 0;
5         e res = {};
6         Kruskal(int n, int m) : e(m), dsu(n) {}
7         template <class C = less<>> void operator()(C && cmp = C()) {
8             sort(e::begin(), e::end(), cmp);
9             for (auto & x : *this) {
10                 if (dsu.merge(x[1], x[2]) != -1) sum += x[0], res.emplace_back(x);
11             }
12         }
13     };

```

Kruskal 重构树：

在跑 Kruskal 的过程中我们会从小到大加入若干条边。现在我们仍然按照这个顺序。

首先新建 n 个集合, 每个集合恰有一个节点, 点权为 0。

每一次加边会合并两个集合, 我们可以新建一个点, 点权为加入边的边权, 同时将两个集合的根节点分别设为新建点的左儿子和右儿子。然后将两个集合和新建点合并成一个集合。将新建点设为根。

不难发现, 在进行 $n - 1$ 轮之后我们得到了一棵恰有 n 个叶子的二叉树, 同时每个非叶子节点恰好有两个儿子。这棵树就叫 Kruskal 重构树。

Kruskal 重构树的性质：

不难发现, 原图中两个点之间的所有简单路径上最大边权的最小值 = 最小生成树上两个点之间的简单路径上的最大值 = Kruskal 重构树上两点之间的 LCA 的权值。

也就是说, 到点 x 的简单路径上最大边权的最小值 $\leq val$ 的所有点 y 均在 Kruskal 重构树上的某一棵子树内, 且恰好为该子树的所有叶子节点。

我们在 Kruskal 重构树上找到 x 到根的路径上权值 $\leq val$ 的最浅的节点。显然这就是所有满足条件的节点所在的子树的根节点。

如果需要求原图中两个点之间的所有简单路径上最小边权的最大值，则在跑 Kruskal 的过程中按边权大到小的顺序加边。

有向图强连通分量

强连通：有向图中任意俩结点连通

连通分量： u 和 v 可相互到达

强联通分量：极大的强连通子图

作用：将有向图缩点(将连通分量缩成一个点)成为一个有向无环图(PAG)(拓扑图)

DFS生成树：

(i) 树枝边：每次搜索找到一个还没有访问过的结点的时候就形成了一条树边

(ii) 前向边：在搜索的时候遇到子树中的结点的时候形成的

(iii) 返祖边：即指向祖先结点的边

(iv) 横叉边：在搜索的时候遇到了一个已经访问过的结点，但是这个结点并不是当前结点的祖先

Tarjan：时间复杂度 $O(n + m)$

(i) dfn 是遍历到 u 的时间戳

(ii) low 是从 u 开始走，所遍历到的最小时间戳是什么

(iii) u 是其所在的强连通分量的最高点 $\Leftrightarrow dfn = low$

(iv) 栈用来记录还未走完的强连通分量

性质：缩点完按照联通分量编号递减一定是拓扑序

结论：拓扑图个数不为1时，将 P 个起点 Q 个终点的拓扑图连成一个强连通分量的最小边数为 $\max(P, Q)$

```
1  template <int len = 2> struct Tarjan_SC : vector<vector<signed>> {
2      vector<array<int, len>> e;
3      vector<signed> dfn, scc, low;
4      vector sc = {};
5      Tarjan_SC(int n, int m) : vector(n), e(m), dfn(n), scc(n), low(n) {}
6      void operator()(stack<signed> s = {}, int tot_dfn = 0) { // !! 会包含 0, 可能 0 会成孤岛
7          vector<signed> vis(size());
8          for (int rt = 0; rt < size(); rt++) {
9              auto self = [&](auto && self, int u) -> void {
10                  s.emplace(u), vis[u] = 1, dfn[u] = low[u] = ++tot_dfn;
11                  for (auto & i : at(u)) {
12                      int v = e[i][0] ^ e[i][1] ^ u;
13                      if (!dfn[v]) self(self, v), low[u] = min(low[u], low[v]);
14                      else if (vis[v]) low[u] = min(low[u], dfn[v]);
15                  }
16                  if (dfn[u] == low[u]) for (sc.emplace_back(); !sc.back().size() || sc.back().back() != u;
17                      s.pop()) {
18                      scc[s.top()] = sc.size() - 1, vis[s.top()] = 0, sc.back().emplace_back(s.top());
19                  }
20                  if (!dfn[rt]) self(self, rt), s = {};
21              }
22          }
23          pair<vector, vector<signed>> Topu_Rebuild() {
24              vector<signed> du(sc.size());
25              vector adj(sc.size());
26              for (int u = 0; u < sc.size(); u++) {
27                  unordered_map<int, int> mp = {{u, 1}};
28                  for (auto & tu : sc[u]) {
29                      for (auto & i : at(tu)) {
30                          int v = scc[e[i][0] ^ e[i][1] ^ tu];
31                          if (!mp.count(v)) adj[u].emplace_back(v), mp[v] = 1, du[v]++;
32                      }
33                  }
34              }
35              return {adj, du};
36          }
37      };
```

无向图双联通分量

边双联通分量： 在一张连通的无向图中,对于两个点 u 和 v 如果无论删去哪条边(只能删去一条)都不能使它们不连通 $\Leftrightarrow u$ 和 v 边双连通

点双联通分量：

在一张连通的无向图中，对于两个点 u 和 v 如果无论删去哪个点(只能删去一个,且不能删 u 和 v 自己)都不能使它们不连通 $\Leftrightarrow u$ 和 v 点双连通

性质：

- (i) 边双连通具有传递性,点双连通不具有传递性
- (ii) 任意两点间存在至少俩条路径 $\Leftrightarrow$ 是边双连通分量
- (iii) 无向图不存在横插边, *Tarjan*时可不用 *in_stack*
- (iv) 孤点也是点双连通分量

差分求桥： 将非树枝边在树上差分,权值为0的为桥,复杂度 $O(n+m)$

差分求割点： 与求桥类似,复杂度 $O(n+m)$

Tarjan求桥： 时间复杂度 $O(n+m)$

若 $low_v > dfn_u \Leftrightarrow u-v$ 为桥,即 $father_v - v$ 为桥

结论： 在只有桥边的图中,有 cnt 个度数为1的点,将图连成一个边双连通分量最小边数为 $\lceil \frac{cnt}{2} \rceil$

```
1  template <int len = 2> struct Tarjan_Bridge : vector<vector<signed>> {
2      vector<array<int, len + 1>> e; // {u, v, is_bridge}
3      vector<signed> dfn, dcc, low;
4      vector dc = {};
5      Tarjan_Bridge(int n, int m) : vector(n), e(m), dfn(n), dcc(n), low(n) {}
6      void operator()(stack<signed> s = {}, int tot_dfn = 0) { // !! 会包含 0, 可能 0 会成孤岛
7          for (int rt = 0; rt < size(); rt++) {
8              auto self = [&](auto && self, int u, int from = -1) -> void { // 用 from 而不用 fa 因为可能重边
9                  s.emplace(u), dfn[u] = low[u] = ++tot_dfn;
10                 for (auto & i : at(u)) {
11                     int v = e[i][0] ^ e[i][1] ^ u;
12                     if (!dfn[v]) {
13                         self(self, v, i), low[u] = min(low[u], low[v]);
14                         if (low[v] > dfn[u] & e[i].back() == true; // 是桥
15                     } else if (i != from) low[u] = min(low[u], dfn[v]);
16                 }
17                 if (dfn[u] == low[u]) for (dc.emplace_back(); !dc.back().size() || dc.back().back() != u;
18             s.pop()) {
19                 dcc[s.top()] = dc.size() - 1, dc.back().emplace_back(s.top());
20             }
21             if (!dfn[rt]) self(self, rt), s = {};
22         }
23     }
24 };
```

Tarjan求割点： 时间复杂度 $O(n+m)$

- (i) 孤立点为单独的点双连通分量
- (ii) 若 $low_u \geq dfn_v$ 且不是根节点时,有一个分支即为割点,否则需要俩个分支

```
1  template <int len = 2> struct Tarjan_cut : vector<vector<signed>> {
2      vector<array<int, len>> e; // {u, v}
3      vector<signed> dfn, low, cut;
4      vector dc = {};
5      Tarjan_cut(int n, int m) : vector(n), e(m), dfn(n), low(n), cut(n) {}
6      void operator()(stack<signed> s = {}, int tot_dfn = 0) { // !!使用时需要去自环, 会包含 0, 0 可能成孤岛
7          for (int rt = 0; rt < size(); rt++) {
8              auto self = [&](auto && self, int u, int cnt = 0) -> void {
9                  s.emplace(u), dfn[u] = low[u] = ++tot_dfn;
10                 if (u == rt && !at(u).size()) dc.emplace_back(1, u); // u 是根节点且没有邻边, 孤岛
11                 for (auto & i : at(u)) {
12                     int v = e[i][0] ^ e[i][1] ^ u;
13                     if (!dfn[v]) {
14                         self(self, v), low[u] = min(low[u], low[v]);
15                         if (low[v] >= dfn[u]) {
16                             if (cnt++, u != rt || cnt > 1) cut[u] = true; // 是割点
17                             for (dc.emplace_back(1, u); dc.back().size() == 1 || dc.back().back() != v; s.pop())
18                                 dc.back().emplace_back(s.top());
19                         }
16                     }
17                 }
18             }
19         }
20     }
21 };
```

```

20         }
21         } else low[u] = min(low[u], dfn[v]);
22     }
23 };
24 if (!dfn[rt]) self(self, rt), s = {};
25 }
26 }
27 };

```

LCA

倍增计算LCA

dfs 时间复杂度 $O(n)$, 单次查询平均复杂度 $O(\log n)$ 最坏为 $O(n)$

```

1  vector<int> dep;
2  vector<array<int, >> fa, cost;
3
4  [&](int n, int rt) {
5      dep = vector<int>(n + 1);
6      fa = cost = vector(n + 1, array<int, >());
7      auto dfs = [&](auto & dfs, int u) -> void {
8          dep[u] = dep[fa[u][0]] + 1;
9          for (int i = 1; i < fa[u].size(); i++) { //倍增算出父亲
10             fa[u][i] = fa[fa[u][i - 1]][i - 1], cost[u][i] = cost[fa[u][i - 1]][i - 1] + cost[u][i - 1];
11         }
12         for (auto & [v, w] : adj[u]) { //dfs
13             if (v == fa[u][0]) continue;
14             fa[v][0] = u, cost[v][0] = w; //记录
15             dfs(dfs, v);
16         }
17     };
18
19     dfs(dfs, rt); //源点
20 } ();
21
22 auto lca = [&](int x, int y, int ans = 0) {
23     if (dep[x] < dep[y]) swap(x, y);
24     for (int h = dep[x] - dep[y], j = 0; h; j++, h >>= 1) { //上升至同一深度
25         if (h & 1) {
26             ans += cost[x][j], x = fa[x][j]; //一定!!! 要让ans在前, 否则x已经改变
27         }
28     }
29     for (int j = fa[x].size() - 1; j >= 0 && x != y; j--) {
30         if (fa[x][j] != fa[y][j]) {
31             ans += cost[x][j], ans += cost[y][j];
32             x = fa[x][j], y = fa[y][j]; //一定!!! 要让ans在前, 否则x已经改变
33         }
34     }
35     return x == y ? array{ans, x} : array{ans + cost[x][0] + cost[y][0], fa[x][0]};
36 };

```

树上差分

作用: 能进行树链上的点权值或边权值进行相加一个值

点差分: 思路: 将 $u \rightarrow v$ 拆成两条链, $u \rightarrow lca, v \rightarrow lca_{son}$

定义差分数组 $d[]$ 为从叶子节点开始差分(为了保证不越界)

对这 $u \rightarrow v$ 这段链的点权加上 $k \Leftrightarrow d_u + = k, d_v + = k, d_{lca} - = k, d_{fa[lca]} - = k$

查询答案从叶子节点累加到 u 为 u 的新权值

边差分: 思路: 由于在边上直接进行差分比较困难, 所以将本来应当累加到红色边上的值向下移动到附近的点里进行点差分

1. 因此 $u \rightarrow v$ 这段链的边加上 $k \Leftrightarrow d_u + = k, d_v + = k, d_{lca} - = k$

树上启发式合并

启发式合并: 让高度小的树成为高度较大的树的子树

树上启发式合并(dsu on tree): 某些子树离线问题更快的解决

e.g.: 给一棵根为1的树, 每次询问子树颜色种类数(树上启发式合并复杂度 $O(n \log n)$)

过程:

每个节点的答案由其子树和其本身得到, 利用这个我们可以先预处理出每个节点子树的大小和它的重儿子, 重儿子同树链剖分一样, 是拥有节点最多子树的儿子。用 $cnt[i]$ 表示颜色 i 的出现次数, $ans[u]$ 记录 u 的答案

(i) 先遍历 u 的轻子树, 并计算答案, 但不保留遍历后它对 cnt 数组的影响

(ii) 遍历它的重儿子, 保留它对 cnt 数组的影响

(iii) 再次遍历 u 的轻儿子的子树结点, 加入这些结点的贡献, 得到 u 的答案

只搜索一遍: 空间复杂度 $O(n)$

```
1 struct Info : gp_hash_table<int, int> {
2     int res;
3     Info(int c) : res(1) { insert({c, 1}); }
4     void merge(Info && dp, int op) { // op 表示是否为链
5         if (size() < dp.size()) std::swap(*this, dp);
6         for (auto & [c, x] : dp) {
7             (*this)[c] += x;
8         }
9         res = size();
10    }
11 };
12
13 struct Info : vector<int> {
14     int r, b, res;
15     Info(char t) : vector(1), r(t == '0'), b(t == '1'), res(0) {}
16     size_t sz() { return size() + r + b; }
17     void merge(Info && dp, int op) { // op 表示是否为链, 维护最短链
18         if (op == 1) return dp.r += r, dp.b += b, std::swap(*this, dp);
19         if (size() == 1) resize(dp.sz()); // 需要更新 lazy
20         else resize(min(size(), dp.sz()));
21         binqueue qb, qr;
22         res = numeric_limits<int>::max();
23         for (int i = 0, t = i - dp.b; i < size(); i++, t++) {
24             int tmp = numeric_limits<int>::max();
25             qb.pop([&](auto & j) { return i - dp.b <= j; });
26             if (0 <= i && i < dp.size()) qb.push({i, dp[i] + i + dp.b});
27             if (qb.size()) tmp = min(tmp, qb.front()[1] - i);
28             qr.pop([&](auto & j) { return i - dp.r - dp.b <= j; });
29             if (0 <= t && t < dp.size()) qr.push({t, dp[t] - t - dp.b});
30             if (qr.size()) tmp = min(tmp, qr.front()[1] + t + dp.b);
31
32             at(i) += tmp;
33             res = min(res, at(i));
34         }
35    }
36 };
37
38 auto dsu_tr = [&](auto && self, int u, int fa = 0) -> Info {
39     Info dp(col[u]);
40     for (auto & v : adj[u]) if (v != fa) {
41         dp.merge(self(self, v, u), adj[u].size() == 1 + !!fa);
42     }
43     return ans[u] = dp.res, move(dp);
44 };
45
46 dsu_tr(dsu_tr, 1);
```

树链剖分

重链剖分: 将树上任意路径划分成不超过 $O(\log n)$ 条连续的链, 每条链的节点 dfn 连续, 方便用数据结构维护信息

(i) 将路径分为不超过 $\log n$ 个连续区间

(ii) 先搜重儿子重链和子树 dfn 连续

```
1 vector<int> fa, sz, big, dfn, dfn_node, dep, top, ta; // top为链顶, ta为重组的 a 数组
2
3 [&](int n, int rt, int totdfn = 0) { // 返回用dfn映射后的权值a
4     fa = big = dfn = dep = top = vector<int>(n + 1), sz = vector<int>(n + 1, 1);
```

```

5
6     auto dfs0 = [&](auto & dfs0, int u) -> void { //这里最好只预处理sz, big和fa,因为搜索顺序需要按dfs的而不是dfs0
7         for (auto & v : adj[u]) {
8             if (v == fa[u]) continue;
9             fa[v] = u, dfs0(dfs0, v);
10            sz[u] += sz[v];
11            if (!big[u] || sz[big[u]] < sz[v]) {
12                big[u] = v;
13            }
14        }
15    };
16
17    auto dfs = [&](auto & dfs, int u, int hi) -> void { //先搜重儿子,记录top,必须在此处计算dfn才是对的
18        dfn[u] = ++totdfn, dfn_node[dfn[u]] = u, dep[u] = dep[fa[u]] + 1, top[u] = hi, ta[dfn[u]] = a[u];
19        if (!big[u]) return;
20        dfs(dfs, big[u], hi);
21        for (auto & v : adj[u]) {
22            if (v == fa[u] || v == big[u]) continue;
23            dfs(dfs, v, v);
24        }
25    };
26
27    dfs0(dfs0, rt), dfs(dfs, rt, rt);
28 } ();
29
30 auto query_path = [&](int u, int v = 0) {
31     vector<array<int, 2>> res; //返回的路径, res.back()[0] 为 lca 的 dfn
32     if (v) { //查询树上路径
33         for (; top[u] != top[v]; u = fa[top[u]]) { //先top[u]和top[v]
34             if (dep[top[u]] < dep[top[v]]) swap(u, v);
35             res.push_back({dfn[top[u]], dfn[u]});
36         }
37         if (dep[u] < dep[v]) swap(u, v); //然后才是u和v
38         res.push_back({dfn[v], dfn[u]});
39     } else { //查询子树
40         res.push_back({dfn[u], dfn[u] + sz[u] - 1});
41     }
42     return res;
43 };
44
45 auto lca = [&](int u, int v) {
46     for (; top[u] != top[v]; u = fa[top[u]]) { //先top[u]和top[v]
47         if (dep[top[u]] < dep[top[v]]) swap(u, v);
48     }
49     return dep[u] < dep[v] ? u : v;
50 };
51
52 auto jump = [&](int u, int k) { //向上跳距离 k
53     if (dep[u] < k) {
54         return (int)-1;
55     }
56     int d = dep[u] - k;
57     while (dep[top[u]] > d) {
58         u = fa[top[u]];
59     }
60     return dfn_node[dfn[u] - dep[u] + d];
61 };
62
63 struct node {
64     int l, r;
65     int sum = 0, add = 0; //需要赋初值, 赋为幺元
66 };
67
68 vector<node> tr;
69
70 auto merge = [&](node l, node r, node u = {}) { //合并贡献
71     u = {l.l, r.r, l.sum + r.sum, u.add}; //记得补全
72     return u;
73 };
74
75 auto pushup = [&](int p) {
76     tr[p] = merge(tr[p << 1], tr[p << 1 | 1], tr[p]); //向上传
77 };
78
79 auto add_down = [&](int k, int p) { //加法的更新

```

```

80     tr[p].add += k, tr[p].sum += (tr[p].r - tr[p].l + 1) * k;
81 };
82
83 auto pushdown = [&](int p) { //add标记下传, 若有乘法和加法, 先计算乘法标记, 且乘法标记会对加法标记有影响
84     add_down(tr[p].add, p << 1), add_down(tr[p].add, p << 1 | 1);
85     tr[p].add = 0;
86 };
87
88 [&](int sz) { //建树
89     tr = vector<node>(sz << 2);
90
91     auto build = [&](auto & build, int l, int r, int p = 1) {
92         tr[p] = {l, r};
93         if (l == r) {
94             tr[p].sum = ;
95             return; //到达叶子节点
96         }
97         pushdown(p); //可不用
98         int mid = l + r >> 1;
99         build(build, l, mid, p << 1), build(build, mid + 1, r, p << 1 | 1);
100        pushup(p);
101    };
102
103    build(build, 1, sz);
104 } ();
105
106 auto update = [&](auto & update, int l, int r, int k, int p = 1) { //更新
107     if (l <= tr[p].l && tr[p].r <= r) {
108         add_down(k, p); //添加标记, 更新节点
109         return;
110     }
111     pushdown(p);
112     if (l <= tr[p << 1].r) update(update, l, r, k, p << 1); //左边有交集
113     if (tr[p << 1 | 1].l <= r) update(update, l, r, k, p << 1 | 1); //右边有交集
114     pushup(p);
115 };
116
117 auto query = [&](auto & query, int l, int r, int p = 1) -> node {
118     if (l <= tr[p].l && tr[p].r <= r) { //被包含
119         return tr[p];
120     }
121     pushdown(p);
122     if (r <= tr[p << 1].r) return query(query, l, r, p << 1); //只在左边
123     if (tr[p << 1 | 1].l <= l) return query(query, l, r, p << 1 | 1); //只在右边
124     return merge(query(query, l, r, p << 1), query(query, l, r, p << 1 | 1)); //左右都有
125     pushup(p); //可不用
126 };

```

拓扑排序

Kahn算法: 记集合 S 为入度为0的点, L 为空

每次从 S 中取出一个点 u 放入 L , 然后把 u 的边删除, 对于边 (u, v) 将该边删除后点 v 的入度为0, 放入 S

最后 L 顶点的顺序为拓扑序的结果

对于图 $G = (V, E)$ 时间复杂度 $O(E + V)$

```

1  vector<vector<int>> G(n), din(n); //存入度
2
3  auto topsort() {
4      vector<int> L;
5      queue<int> S;
6      for (int i = 1; i <= n; i++) {
7          if (din[i] == 0) {
8              S.push(i);
9          }
10     }
11     while (S.size()) {
12         int u = S.front();
13         S.pop();
14         L.push_back(u);
15         for (auto v : G[u]) {
16             if (--din[v] == 0) {

```

```

17         S.push(v);
18     }
19 }
20 }
21 };

```

2-SAT

```

1  struct TwoSat {
2      int n;
3      std::vector<std::vector<int>> e;
4      std::vector<bool> ans;
5      TwoSat(int n) : n(n), e(2 * n), ans(n) {}
6      void addClause(int u, bool f, int v, bool g) {
7          e[2 * u + !f].push_back(2 * v + g);
8          e[2 * v + !g].push_back(2 * u + f);
9      }
10     bool satisfiable() {
11         std::vector<int> id(2 * n, -1), dfn(2 * n, -1), low(2 * n, -1);
12         std::vector<int> stk;
13         int now = 0, cnt = 0;
14         std::function<void(int)> tarjan = [&](int u) {
15             stk.push_back(u);
16             dfn[u] = low[u] = now++;
17             for (auto v : e[u]) {
18                 if (dfn[v] == -1) {
19                     tarjan(v);
20                     low[u] = std::min(low[u], low[v]);
21                 } else if (id[v] == -1) {
22                     low[u] = std::min(low[u], dfn[v]);
23                 }
24             }
25             if (dfn[u] == low[u]) {
26                 int v;
27                 do {
28                     v = stk.back();
29                     stk.pop_back();
30                     id[v] = cnt;
31                 } while (v != u);
32                 ++cnt;
33             }
34         };
35         for (int i = 0; i < 2 * n; ++i) if (dfn[i] == -1) tarjan(i);
36         for (int i = 0; i < n; ++i) {
37             if (id[2 * i] == id[2 * i + 1]) return false;
38             ans[i] = id[2 * i] > id[2 * i + 1];
39         }
40         return true;
41     }
42     std::vector<bool> answer() { return ans; }
43 };
44

```

欧拉图

定义:

1. **欧拉回路**: 通过图中每条边恰好一次的回路
2. **欧拉通路**: 通过图中每条边恰好一次的通路
3. **欧拉图**: 存在欧拉回路的图
4. **半欧拉图**: 具有欧拉通路但不具有欧拉回路的图

性质:

1. 无向图是欧拉图 $\Leftrightarrow$ 非零顶点是连通的 $\wedge$ 顶点的度数都是偶数
2. 无向图是半欧拉图 $\Leftrightarrow$ 非零顶点是联通的 $\wedge$ 恰有 2 个奇数顶点
3. 有向图是欧拉图 $\Leftrightarrow$ 非零顶点是 (强) 联通的 $\wedge$ 每个顶点的入度等于出度
4. 有向图是半欧拉图 $\Leftrightarrow$ 非零顶点是 (弱) 联通的 $\wedge$ 至多一个顶点的出度 - 入度 = 1 $\wedge$ 至多一个顶点的入度 - 出度 = 1 $\wedge$ 其他顶点出度等于入度

```

1 // 返回为 0 为欧拉回路, > 0 为欧拉路径, < 0 为不存在, op 为 0 表示无向, 为 1 表示有向
2 auto [euler_type, euler_path] = [&](int n, int m, int op, vector<vector<array<int, 2>>> adj) {
3     vector<int> vis(m + 1), res; // 有向时不需要 vis, 无向不需要 indu
4     int flag = 0; // 若flag 最后为 0, 则为欧拉回路
5     for (int i = 1; i <= n; i++) { // adj 最后一维记录的为边的编号
6         if (!op && (adj[i].size() & 1)) {
7             if (++flag > 2) return pair{(int)-1, res};
8         } if (op && adj[i].size() != indu[i]) {
9             switch ((int)adj[i].size() - indu[i]) {
10                case 1 : flag += 1; break;
11                case -1 : flag += 3; break;
12                default : return pair{(int)-1, res};
13            }
14            if (!(flag & 1) && flag != 4) return pair{(int)-1, res};
15        }
16        // sort(adj[i].begin(), adj[i].end(), greater()); // 按字典序从小到大
17    }
18    for (int i = 1; i <= n; i++) {
19        auto dfs = [&](auto & dfs, int u) -> void {
20            while (adj[u].size()) {
21                auto [v, i] = adj[u].back();
22                adj[u].pop_back();
23                if (vis[i]) continue; // 有向时不需要
24                vis[i] = 1;
25                dfs(dfs, v);
26            }
27            res.push_back(u); // 回溯 push, 为了保证其他环都跑完
28        };
29        if (!flag && adj[i].size()) { dfs(dfs, i); break; }
30        if (!op && (adj[i].size() & 1)) { dfs(dfs, i); break; }
31        if (op && adj[i].size() > indu[i]) { dfs(dfs, i); break; }
32    }
33    if (accumulate(adj.begin() + 1, adj.end(), 0, [&](auto now, auto & x) { return now + x.size(); })) {
34        return pair{(int)-1, res}; // 边不联通
35    }
36    return pair{flag, vector(res.rbegin(), res.rend())}; // reverse 因为是回溯的时候 push
37 } ();

```

二分图

二分图判定： 染色法

二分图最大匹配：

- **匈牙利算法：** 时间复杂度 $O(nm)$

```

1 vector<int> mch;
2
3 int ans = [&](int n, int m, int ans = 0) {
4     mch = vector<int>(n + m + 1);
5     vector<int> vis(n + m + 1);
6
7     for (int i = 1; i <= n; i++) {
8         fill(vis.begin() + n + 1, vis.end(), 0);
9
10        auto dfs = [&](auto & dfs, int u) -> bool {
11            for (auto & v : adj[u]) {
12                if (vis[v]) continue;
13                vis[v] = 1;
14                if (!mch[v] || dfs(dfs, mch[v])) {
15                    mch[u] = v, mch[v] = u;
16                    return true;
17                }
18            }
19            return false;
20        };
21
22        ans += dfs(dfs, i);
23    }
24
25    return ans;
26 } ();

```

- **Hopcroft-Karp算法：** 时间复杂度 $O(m\sqrt{n})$

```

1  vector<int> mch;
2
3  int ans = [&](int n, int m, int ans = 0) {
4      mch = vector<int>(n + m + 1);
5      vector<int> vis(n + m + 1);
6
7      while (1) {
8          queue<int> q;    //bfs找到最多最短增广路。
9          int mx_dis = 1e9;
10
11         vector<int> dis = vector<int>(n + m + 1, 1e9);
12
13         for (int i = 1; i <= n; i++) {
14             if (!mch[i]) {
15                 q.push(i);
16                 dis[i] = 1;
17             }
18         }
19
20         while (q.size()) {
21             int u = q.front();
22             q.pop();
23
24             if (dis[u] > mx_dis) break;
25
26             for (auto & v : adj[u]) {
27                 if (dis[v] == 1e9) {    //右边的v点还未遍历
28                     dis[v] = dis[u] + 1;
29                     if (!mch[v]) {
30                         mx_dis = dis[v];
31                     } else {
32                         dis[mch[v]] = dis[v] + 1;
33                         q.push(mch[v]);
34                     }
35                 }
36             }
37         }
38         if (mx_dis == 1e9) break;    //无增广路
39         fill(vis.begin() + n + 1, vis.end(), 0);
40
41         for (int i = 1; i <= n; i++) {    //遍历每条增广路
42             auto dfs = [&](auto & dfs, int u) -> bool {
43                 for (auto & v : adj[u]) {
44                     if (vis[v] || dis[v] != dis[u] + 1) continue;    //沿着增广路的边走
45                     vis[v] = 1;
46                     if (mch[v] && dis[v] == mx_dis) continue;    //达到最长长度不能再增广了
47                     if (!mch[v] || dfs(dfs, mch[v])) {
48                         mch[u] = v, mch[v] = u;
49                         return true;
50                     }
51                 }
52                 return false;
53             };
54             if (!mch[i]) {    //这里匈牙利不用，但是hk需要
55                 ans += dfs(dfs, i);
56             }
57         }
58     }
59
60     return ans;
61 } ();

```

网络流

```

1  template<class type = int>
2  struct Dinic : vector<vector<int>> {    // 0(n^2 * m), 二分图最大匹配 0(n^0.5 * m)
3      struct Edge {
4          int v;
5          type c;
6          Edge(int v, type c) : v(v), c(c) {}
7      };
8
9      vector<Edge> e;

```

```

10     vector<int> cur, h;
11     Dinic(int n) : vector(n) {}
12     void add(int u, int v, type c) {
13         at(u).emplace_back(e.size(), e.emplace_back(v, c));
14         at(v).emplace_back(e.size(), e.emplace_back(u, 0));
15     }
16
17     int bfs(int & s, int & t) {
18         h.assign(size(), numeric_limits<int>::max());
19         queue<int> q;
20         h[s] = 0, q.emplace(s);
21         while (q.size()) {
22             int u = q.front(); q.pop();
23             for (auto & i : at(u)) {
24                 auto & [v, c] = e[i];
25                 if (c > 0 && h[v] > h[u] + 1) h[v] = h[u] + 1, q.emplace(v);
26             }
27         }
28         return h[t] != numeric_limits<int>::max();
29     }
30     type dfs(int u, int & t, type flow) {
31         if (u == t) return flow;
32         type ret = flow, a;
33         for (int & i = cur[u]; i < at(u).size(); i++) {
34             auto & [v, c] = e[at(u)[i]];
35             if (c > 0 && h[v] == h[u] + 1 && (a = dfs(v, t, min(ret, c)))) {
36                 ret -= a, c -= a, e[at(u)[i] ^ 1].c += a;
37                 if (ret == 0) return flow;
38             }
39         }
40         return flow - ret;
41     }
42     type maxflow(int s, int t, type flow = 0) {
43         while (bfs(s, t)) {
44             cur.assign(size(), 0), flow += dfs(s, t, numeric_limits<type>::max());
45         }
46         return flow;
47     }
48 };
49
50 template<class type = int>
51 struct ISAP : vector<vector<int>> { // O(n^2 * m)
52     struct Edge {
53         int v;
54         type c;
55         Edge(int v, type c) : v(v), c(c) {}
56     };
57
58     vector<Edge> e;
59     vector<int> cur, h, gap;
60     ISAP(int n) : vector(n) {}
61     void add(int u, int v, type c) {
62         at(u).emplace_back(e.size(), e.emplace_back(v, c));
63         at(v).emplace_back(e.size(), e.emplace_back(u, 0));
64     }
65
66     void bfs(int & t) { // 反向bfs
67         h.assign(size(), numeric_limits<int>::max()), gap.assign(size() + 2, 0);
68         queue<int> q;
69         h[t] = 0, gap[h[t]]++, q.emplace(t);
70         while (q.size()) {
71             int u = q.front(); q.pop();
72             for (auto & i : at(u)) {
73                 auto & [v, c] = e[i];
74                 if (e[i ^ 1].v && h[v] > h[u] + 1) h[v] = h[u] + 1, gap[h[v]]++, q.emplace(v);
75             }
76         }
77     }
78     type dfs(int u, int & s, int & t, type flow) {
79         if (u == t) return flow;
80         type ret = flow, a;
81         for (int & i = cur[u]; i < at(u).size(); i++) {
82             auto & [v, c] = e[at(u)[i]];
83             if (c > 0 && h[v] + 1 == h[u] && (a = dfs(v, s, t, min(ret, c)))) {
84                 ret -= a, c -= a, e[at(u)[i] ^ 1].c += a;

```

```

85         if (ret == 0) return flow;
86     }
87 }
88 if (--gap[h[u]] == 0) h[s] = size();
89 return gap[+h[u]]++, flow - ret;
90 }
91 type maxflow(int s, int t, type flow = 0) {
92     for (bfs(t); h[s] < size(); ) {
93         cur.assign(size(), 0), flow += dfs(s, s, t, numeric_limits<type>::max());
94     }
95     return flow;
96 }
97 };
98
99 template<class type = int>
100 struct HLPP : vector<vector<int>>> { // O(n^2 * sqrt(m))
101     struct Edge {
102         int v;
103         type c;
104         Edge(int v, type c) : v(v), c(c) {}
105     };
106
107     vector<Edge> e;
108     vector<int> h, gap;
109     int level = 0;
110     vector<type> ex;
111     vector<stack<int>> B;
112     HLPP(int n) : vector(n) {}
113     void add(int u, int v, type c) {
114         at(u).emplace_back(e.size(), e.emplace_back(v, c));
115         at(v).emplace_back(e.size(), e.emplace_back(u, 0));
116     }
117
118     int bfs(int & s, int & t) { // 反向bfs
119         h.assign(size(), numeric_limits<int>::max()), gap.assign(size() + 2, 0);
120         queue<int> q;
121         h[t] = 0, gap[h[t]]++, q.emplace(t);
122         while (q.size()) {
123             int u = q.front(); q.pop();
124             for (auto & i : at(u)) {
125                 auto & [v, c] = e[i];
126                 if (e[i ^ 1].v && h[v] > h[u] + 1) h[v] = h[u] + 1, gap[h[v]]++, q.emplace(v);
127             }
128         }
129         return h[s] != numeric_limits<int>::max();
130     }
131
132     int push(int u, int & s, int & t) {
133         int init = u == s;
134         for (auto & i : at(u)) {
135             auto & [v, c] = e[i];
136             if (!init && h[u] != h[v] + 1 || !c || h[v] == numeric_limits<int>::max()) continue;
137             type k = init ? c : min(c, ex[u]);
138             if (v != s && v != t && !ex[v]) B[h[v]].push(v), level = max(level, h[v]);
139             ex[u] -= k, ex[v] += k, c -= k, e[i ^ 1].c += k;
140             if (!ex[u]) return 0;
141         }
142         return 1;
143     }
144
145     void relabel(int u) {
146         h[u] = numeric_limits<int>::max();
147         for (auto & i : at(u)) {
148             if (e[i].c) h[u] = min(h[u], h[e[i].v]);
149         }
150         if (++h[u] < size()) B[h[u]].push(u), level = max(level, h[u]), ++gap[h[u]];
151     }
152
153     int slect() {
154         for (; level > -1 && B[level].size() == 0; level--);
155         return level == -1 ? 0 : B[level].top();
156     }
157
158     type maxflow(int s, int t, type flow = 0) {
159         if (!bfs(s, t)) return 0;
160         ex.assign(size(), 0), B.assign(size(), stack<int>());
161         h[s] = size(), push(s, s, t);
162         for (int u = 0; u = slect(); ) {
163             B[level].pop();

```

```

160         if (push(u, s, t)) {
161             if (--gap[h[u]] == 0) for (int i = 0; i < size(); i++) {
162                 if (i != s && h[i] > h[u] && h[i] < size() + 1) h[i] = size() + 1;
163             }
164             relabel(u);
165         }
166     }
167     return ex[t];
168 }
169 };
170
171 template <class type = int>
172 struct MCMF : vector<vector<int>>> {
173     struct Edge {
174         int v;
175         type c, f; // 容量, 费用
176         Edge(int v, type c, type f) : v(v), c(c), f(f) {}
177     };
178
179     vector<Edge> e;
180     vector<type> h, dis;
181     vector<int> pre;
182     MCMF(int n) : vector(n) {}
183     void add(int u, int v, type c, type f) {
184         at(u).emplace_back(e.size(), e.emplace_back(v, c, f));
185         at(v).emplace_back(e.size(), e.emplace_back(u, 0, -f));
186     }
187
188     int dijkstra(int & s, int & t) {
189         dis.assign(size(), numeric_limits<type>::max()), pre.assign(size(), -1);
190         std::priority_queue<pair<type, int>, vector<pair<type, int>>, greater<>> q;
191         dis[s] = 0, q.emplace(0, s);
192         while (q.size()) {
193             auto [d, u] = q.top(); q.pop();
194             if (dis[u] < d) continue;
195             for (auto & i : at(u)) {
196                 auto & [v, c, f] = e[i];
197                 if (c > 0 && dis[v] > d + h[u] - h[v] + f) {
198                     dis[v] = d + h[u] - h[v] + f, pre[v] = i;
199                     q.emplace(dis[v], v);
200                 }
201             }
202         }
203         return dis[t] != numeric_limits<type>::max();
204     }
205
206     void spfa(int & s, int & t) { // 存在负权边
207         h.assign(size(), numeric_limits<type>::max());
208         vector<int> vis(size());
209         queue<int> q;
210         h[s] = 0, vis[s] = 1, q.push(s);
211         while (q.size()) {
212             int u = q.front(); q.pop(), vis[u] = 0;
213             for (auto & i : at(u)) {
214                 auto & [v, c, f] = e[i];
215                 if (c > 0 && h[v] > h[u] + f) {
216                     h[v] = h[u] + f;
217                     if (!vis[v]) vis[v] = 1, q.push(v);
218                 }
219             }
220         }
221     }
222
223     array<type, 2> mncmf(int s, int t, int flag, type flow = 0, type cost = 0) {
224         if (flag) spfa(s, t); // 若存在负权边, 也可以使用消负权法
225         else h.assign(size(), 0); // 若全为正权值
226         while (dijkstra(s, t)) {
227             type a = numeric_limits<type>::max();
228             for (int i = 0; i < size(); h[i++] += dis[i]);
229             for (int i = t; i != s; a = min(a, e[pre[i]].c), i = e[pre[i] ^ 1].v);
230             for (int i = t; i != s; i = e[pre[i] ^ 1].v) {
231                 e[pre[i]].c -= a, e[pre[i] ^ 1].c += a;
232             }
233             flow += a, cost += a * h[t];
234         }
235         return {flow, cost};
236     }
237 }

```

概念:

- **流网络**: $G = (V, E)$ 假定不存在反向边
- **可行流**: $f = \begin{cases} \text{容量限制} \Leftrightarrow 0 \leq f(u, v) \leq c(u, v) \\ \text{流量守恒} \Leftrightarrow \forall x \in V, \sum_{(V, x) \in E} f(V, x) = \sum_{(x, V) \in E} f(x, V) \end{cases}$
 - 流量值 $|f| = \sum_{(S, V) \in E} f(S, V) - \sum_{(V, S) \in E} f(V, S)$
 - 最大流: 最大可行流
- **残留网络**: $G_f = (V, E \text{ 和 } E \text{ 的所有反向边})$, $c'(u, v) = \begin{cases} c(u, v) - f(u, v) & (u, v) \in E \\ f(v, u) & (v, u) \in E \end{cases}$
 - **意义**: 从源点出发已经无法找到一条增广路到底汇点, 因为所有的路已经被割了
 - 考虑反向边, 残留网络的可行流 $f' +$ 原图的可行流 = 原图的另一个可行流
 - $|f' + f| = |f'| + |f|$
 - $|f'|$ 可能为负数
- **增广路径**: 从源点起残留网络中沿着容量大于0的边走到汇点的路径
- **割**: 将 V 分成 $[S, T]$, $S \cup T = V, S \cap T = \emptyset, s \in S, t \in T$
 - **割的容量(不考虑反向边)**: $c(S, T) = \sum_{u \in S} \sum_{v \in T} c(u, v)$, 容量最小被称为最小割 = 最大流
 - **割的流量(考虑反向边)**: $f(S, T) = \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{u \in S} \sum_{v \in T} f(v, u)$
 - $\forall [S, T], f(S, T) = |f| \leq c(S, T)$, 即最大流小于最小割的流量
- **最大流最小割定理**: 可行流 f 是最大流 $\Leftrightarrow$ 可行流 f 的残留网络不存在增广路 $\Leftrightarrow \exists [S, T], s. t. |f| = c(S, T)$

EK算法: 时间复杂度 $O(nm^2)$, 点加边在1000到10000

Dinic算法: 时间复杂度 $O(n^2m)$, 二分图 $O(m\sqrt{n})$, 点加边在10000到100000

(i) bfs 出层次图

(ii) dfs 出阻塞流, 起点 $S \rightarrow$ 汇点 T

注: 解决二分图最大匹配, 将起点连二分图一边, 汇点连另一边, 最大流为最大匹配, 复杂度 $O(m\sqrt{n})$

问题模型 1:

有 n 个物品和两个集合 A, B , 如果一个物品没有放入 A 集合会花费 a_i , 没有放入 B 集合会花费 b_i ; 还有若干个形如 u_i, v_i, w_i 限制条件, 表示如果 u_i 和 v_i 同时不在一个集合会花费 w_i 。每个物品必须且只能属于一个集合, 求最小的代价。

这是一个经典的 "二者选其一" 的最小割题目。我们对于每个集合设置源点 s 和汇点 t , 第 i 个点由 s 连一条容量为 a_i 的边、向 t 连一条容量为 b_i 的边。对于限制条件 u, v, w , 我们在 u, v 之间连容量为 w 的双向边。

注意到当源点和汇点不相连时, 代表这些点都选择了其中一个集合。如果将连向 s 或 t 的边割开, 表示不放在 A 或 B 集合, 如果把物品之间的边割开, 表示这两个物品不放在同一个集合。

最小割就是最小花费。

问题模型 2:

问题模型一限制改为, u 和 v 在同一个集合有花费 w , 求最大花费。

使用模型二解决: 先全部放 A 集合计算花费为 $\sum_{i=1} b_i + \sum w$, 给每个点权值为 $a_i - b_i - W$ (W 为 i, v 在同一个集合的花费) 表示从 A 移动到 B 的花费, 将 u, v 在同一个集合的花费建成虚点连单向边到 u, v 权值为 w , 作为两个点都被选扣除俩倍的的补偿, 对图求最大权独立集即为最大花费

最大权值闭合图, 即给定一张有向图, 每个点都有一个权值 (可以为正或负或 0), 你需要选择一个权值和最大的子图, 使得子图中每个点的后继都在子图中。

做法: 建立超级源点 s 和超级汇点 t , 若节点 u 权值为正, 则 s 向 u 连一条有向边, 边权即为该点点权; 若节点 u 权值为负, 则由 u 向 t 连一条有向边, 边权即为该点点权的相反数。原图上所有边权改为 ∞ 。跑网络最大流, 将所有正权值之和减去最大流, 即为答案。

几个小结论来证明:

1. 每一个符合条件的子图都对应流量网络中的一个割。因为每一个割将网络分为两部分, 与 s 相连的那部分满足没有边指向另一部分, 于是满足上述条件。这个命题是充要的。
2. 最小割所去除的边必须与 s 和 t 其中一者相连。因为否则边权是 ∞ , 不可能成为最小割。
3. 我们所选择的那部分子图, 权值和 = 所有正权值之和 - 我们未选择的正权值点的权值之和 + 我们选择的负权值点的权值之和。当我们不选择一个正权值点时, 其与 s 的连边会被断开; 当我们选择一个负权值点时, 其与 t 的连边会被断开。断开的边的边权之和即为割的容量。于是上述式子转化为:

$$\text{权值和} = \text{所有正权值之和} - \text{割的容量}$$

于是得出结论, 最大权值和 = 所有正权值之和 - 最小割 = 所有正权值之和 - 最大流。

最大流之上下界可行流

无源汇上下界可行流(循环流): 构造类似的图 (G', f') , s. t. 原图 $(G, f) \Leftrightarrow$ 新图 (G', f')

- **容量上下界:**

$C_l(u, v) \leq f(u, v) \leq C_r(u, v) \Leftrightarrow 0 \leq f(u, v) - C_l(u, v) \leq C_r(u, v) - C_l(u, v)$, 满足容量限制, 但不一定满足原图中流量守恒

- **流量守恒:** 假设一个点初始流入流量减初始流出流量为 M

- $M = 0 \Leftrightarrow$ 流量平衡
- $M > 0 \Leftrightarrow$ 入流量过大时, 相当于新图流入变少, 需要新增源点 S' , S' 向其连流量为 M 的边来使流量平衡
- $M < 0 \Leftrightarrow$ 出流量过大时, 相当于新图流入过多, 需要新增汇点 T' , 其向 T' 连流量为 $-M$ 的边

- **是否存在可行流:** 跑 S' 到 T' 的最大流, 需要 S' 连出的边全部流满, 即流量守恒 $\Leftrightarrow$ 存在可行流

```
1 void QAQ() {
2     int n, m;
3     cin >> n >> m;
4     int S = 0, T = n + 1;
5     Dinic adj(T + 1);
6     vector<int> in(n + 1), low(m);
7
8     for (int i = 0; i < m; i++) {
9         int u, v, c;
10        cin >> u >> v >> low[i] >> c;
11        adj.add(u, v, c - low[i]), in[u] -= low[i], in[v] += low[i];
12    }
13    int tot = 0;
14    for (int i = 1; i <= n; i++) {
15        if (in[i] > 0) tot += in[i], adj.add(S, i, in[i]);
16        else if (in[i] < 0) adj.add(i, T, -in[i]);
17    }
18
19    if (adj.maxflow(S, T) < tot) cout << "NO\n";
20    else {
21        cout << "YES\n";
22        for (int i = 0; i < m; i++) {
23            cout << adj.e[i << 1 | 1].c + low[i] << "\n";
24        }
25    }
26 }
```

有源汇上下界可行流(最大流,最小流):

可行流: 不满足流量守恒, 加入一条 T 到 S 的上界为 ∞ 的边, 下界为0的边转化为无源汇上下界可行流

- **有解条件:** 满足无源汇上下界可行流有解条件 $\Leftrightarrow S$ 到 T 的可行流流量等于 T 到 S 的附加边的流量

跑 S' 到 T' 最大流后, 记 $f_0(S, T)$ 为此最大流中 S 到 T 的流, $f'(S, T)$ 为此图残留网络中 S 到 T 的流

由于 S' 的边全部流满不影响 S 到 T (此新图特殊), 原图 S 到 T 的任意可行流 $f(S, T) = f_0(S, T) + f'(S, T)$

最大流: 即在残留网络中跑 S 到 T 的最大流 $f'_{max}(S, T)$ 与 S' 到 T' 最大流中 S 到 T 的流 $f_0(S, T)$ 相加

最小流: 即在残留网络中跑 T 到 S 的最大流 $f'_{min}(T, S) = f'_{min}(S, T)$ 与 $f_0(S, T)$ 相加

```
1 void QAQ() {
2     int n, m, s, t;
3     cin >> n >> m >> s >> t;
4     int S = 0, T = n + 1;
5     Dinic adj(T + 1);
6     vector<int> in(n + 1), low(m);
7
8     for (int i = 0; i < m; i++) {
9         int u, v, c;
10        cin >> u >> v >> low[i] >> c;
11        adj.add(u, v, c - low[i]), in[u] -= low[i], in[v] += low[i];
12    }
13    int tot = 0;
14    for (int i = 1; i <= n; i++) {
15        if (in[i] > 0) tot += in[i], adj.add(S, i, in[i]);
16        else if (in[i] < 0) adj.add(i, T, -in[i]);
17    }
18    adj.add(t, s, numeric_limits<int>::max());
19
20    if (adj.maxflow(S, T) < tot) cout << "No Solution\n";
21    else {
```

```

22     int maxflow = adj.e.back().c;    // 即 s -> t 的实际流量
23     adj[s].pop_back(), adj[t].pop_back(), adj.e.pop_back(), adj.e.pop_back();    // 删除 t -> s
24     cout << maxflow + adj.maxflow(s, t) << "\n";    // 最大流
25     cout << maxflow - adj.maxflow(t, s) << "\n";    // 最小流
26 }
27 }

```

最大流之技巧

多源汇最大流：建立超级源点 S 连向所有源点,容量为 ∞ ,建立超级汇点 T ,所有汇点连向 T ,容量为 ∞

关键边（割边）：只给其扩大容量之后流网络的最大流增加的边为关键边

在残留网络中,沿着剩余容量大于0的边走,对源点能到达的所有点打标记,对汇点能到达的所有点打标记,关键边为剩余容量为0且能到达源汇点

最大流之最大流判定：可直接二分加最大流,或枚举过程建立新边不断在残留网络跑最大流(即分层建图)

最大流之拆点：当结有限制次数等其他限制,可拆一个点为多个点,中间连边,容量为限制

最大流之最小割之平面图转最短路：在平面图中,最小割 $\Leftrightarrow$ 沿着最小割轮廓的最短路

最小割之最大权闭合图、最大密度子图

最小割之最大权闭合图：一张有向图,每个点都有一个权值,权值和最大的子图,且子图中每个点的后继都在子图中

做法：建立源点 s 和汇点 t ,若 u 权为正,则 s 向 u 连一条有向边,边权即为点权,若 u 权为负,则由 u 向 t 连一条有向边,边权即为点权的相反数,原图上所有边权改为 ∞

跑网络最大流,将所有正权值之和减去最大流,即为答案

证明： $\forall$ 闭合子图, $\exists$ 割, s, t . 闭合子图与割一一对应,记此割为 $C[S, T] = C, V_+$ 为点权为正点集, V_- 为点权为负点集, V 为闭合子图

$$C[S, T] = \sum_{x \in V_+} W_x + \sum_{x \in V_-} -W_x$$

$$W(V) = \sum_{x \in V_+} W_x + \sum_{x \in V_-} W_x$$

$$C[S, T] + W(V) = \sum_{x \in V_+} W_x \Rightarrow \text{当 } C[S, T] \text{ 最小时, } W(V) \text{ 最大}$$

最小割之最大密度子图：设一张无向图 $G = (V, E)$,对于子图 $G' = (V', E')$,密度 $D = \frac{|E'|}{|V'|}$,密度 D 最大的子图为最大密度子图

思路：用01分数规划设 $g = D$,即二分 $g|V'| - |E'|$

做法：

V 中点间连容量为1的边, V 中的点 u 向汇点 t 连容量 $2g - d_u + U$ 的边(U 为保证容量非负,取 $U = |E|$),源点 s 向 V 中的点 u 连接容量 U 的边

跑网络最大流,二分答案加最小割, S 所在集合为最大密度子图

证明：注意到 $\forall$ 割,只由 $s \rightarrow \overline{V'}$, $V' \rightarrow t$, 和 $V' \rightarrow \overline{V'}$ 构成

$$\Rightarrow c[S, T] = \sum_{u \in \overline{V'}} U + \sum_{u \in V'} (2g - d_u + U) + \sum_{u \in V'} \sum_{v \in \overline{V'}} c(u, v)$$

$$\text{由于 } \sum_{u \in V'} (-d_u) + \sum_{u \in V'} (2g - d_u + U) + \sum_{u \in V'} \sum_{v \in \overline{V'}} c(u, v) = -2|E'|$$

$$\Rightarrow c[S, T] = |V|U - 2|E'| + \sum_{u \in V'} (2g) = |V|U + 2(g|V'| - |E'|)$$

最小割(二分图)之最小权覆盖集、最大权独立集

二分图的最小权点覆盖集：选点权和最少的点,满足每条边至少有一个端点被选

做法： $\forall$ 二分图,源点向一排节点连容量为点权的边,另一排节点向汇点连容量为点权的边,原图的所有边设置容量为 ∞ ,方向为 S 到 T

跑最大流即为最小权点覆盖

二分图的最大权独立集：选点权和最多的点,满足两两之间没有边相连,是最小权点覆盖的补集

做法：最大独立全集 $= \sum w - \text{最小点权覆盖}$

特殊：在二分图中,当点权为1时,最小点覆盖 $=$ 最大匹配 $= n - \text{最大独立集}$

最小费用最大流

Dinic算法 + SPFA: 时间复杂度最坏 $O(nmf)$, f 为最大流

(i) 用最大流的思路, 将 bfs 分层换成 $SPFA$ 求最短路的分层(由于方向边可能为负, 暂时不用 $Dijkstra$)

上下界费用流

与上下界可行流类似, 但记得下届加费用的贡献!!

使用上下界费用流限制负权边全部流满, 将负权转化为正权, 可以做到消除负圈

注意, 本题中允许一个不经过 s, t 的环整体加上一个流量

```
1 void QAQ() {
2     int n, m, s, t;
3     cin >> n >> m >> s >> t;
4     int S = 0, T = n + 1;
5
6     MCMF adj(T + 1);
7     vector<int> in(n + 1);
8     int mincost = 0;
9
10    for (int i = 1; i <= m; i++) {
11        int u, v, c, f;
12        cin >> u >> v >> c >> f;
13        if (f < 0) {
14            adj.add(v, u, c, -f);
15            in[u] -= c, in[v] += c, mincost += c * f;
16        } else adj.add(u, v, c, f);
17    }
18    for (int i = 1; i <= n; i++) {
19        if (in[i] > 0) adj.add(S, i, in[i], 0);
20        else if (in[i] < 0) adj.add(i, T, -in[i], 0);
21    }
22
23    adj.add(t, s, numeric_limits<int>::max(), 0);
24
25    mincost += adj.mnccmf(S, T, 0)[1];
26    int maxflow = adj.e.back().c; // s -> t 实际流量
27    adj[s].pop_back(), adj[t].pop_back(), adj.e.pop_back(), adj.e.pop_back(); // 删除 t -> s
28    auto [flow, cost] = adj.mnccmf(s, t, 0);
29    cout << maxflow + flow << " " << mincost + cost << "\n";
30 }
```

Prüfer 序列

对树建立 Prüfer 序列

Prüfer 是这样建立的: 每次选择一个编号最小的叶结点并删掉它, 然后在序列中记录下它连接到的那个结点。重复 $n - 2$ 次后就只剩下两个结点, 算法结束。

显然使用堆可以做到 $O(n \log n)$ 的复杂度

```
1 // 代码摘自原文, 结点是从 0 标号的
2 vector<vector<int>> adj;
3
4 vector<int> pruefer_code() {
5     int n = adj.size();
6     set<int> leafs;
7     vector<int> degree(n);
8     vector<bool> killed(n, false);
9     for (int i = 0; i < n; i++) {
10         degree[i] = adj[i].size();
11         if (degree[i] == 1) leafs.insert(i);
12     }
13
14     vector<int> code(n - 2);
15     for (int i = 0; i < n - 2; i++) {
16         int leaf = *leafs.begin();
17         leafs.erase(leafs.begin());
18         killed[leaf] = true;
```

```

19     int v;
20     for (int u : adj[leaf])
21         if (!killed[u]) v = u;
22     code[i] = v;
23     if (--degree[v] == 1) leafs.insert(v);
24 }
25 return code;
26 }

```

Prüfer 序列的线性构造算法:

线性构造的本质就是维护一个指针指向我们即将删除的结点。首先发现，叶结点数是非严格单调递减的，删去一个叶结点，叶结点总数要么不变要么减 1。

于是我们考虑这样一个过程：维护一个指针 p 。初始时 p 指向编号最小的叶结点。同时我们维护每个结点的度数，方便我们知道在删除结点的时候是否产生新的叶结点。操作如下：

1. 删除 p 指向的结点，并检查是否产生新的叶结点。
2. 如果产生新的叶结点，假设编号为 x ，我们比较 p 和 x 的大小关系。如果 $x > p$ ，那么不做其他操作；否则就立刻删除 x ，然后检查删除 x 后是否产生新的叶结点，重复步骤 2，直到未产生新节点或者新节点的编号 $> p$ 。
3. 让指针 p 自增直到遇到一个未被删除的叶结点为止。

```

1 // 从原文摘的代码，同样以 0 为起点
2 vector<vector<int>>> adj;
3 vector<int> parent;
4
5 void dfs(int v) {
6     for (int u : adj[v]) {
7         if (u != parent[v]) parent[u] = v, dfs(u);
8     }
9 }
10
11 vector<int> pruefer_code() {
12     int n = adj.size();
13     parent.resize(n), parent[n - 1] = -1;
14     dfs(n - 1);
15
16     int ptr = -1;
17     vector<int> degree(n);
18     for (int i = 0; i < n; i++) {
19         degree[i] = adj[i].size();
20         if (degree[i] == 1 && ptr == -1) ptr = i;
21     }
22
23     vector<int> code(n - 2);
24     int leaf = ptr;
25     for (int i = 0; i < n - 2; i++) {
26         int next = parent[leaf];
27         code[i] = next;
28         if (--degree[next] == 1 && next < ptr) {
29             leaf = next;
30         } else {
31             ptr++;
32             while (degree[ptr] != 1) ptr++;
33             leaf = ptr;
34         }
35     }
36     return code;
37 }

```

Prüfer 序列的性质:

1. 在构造完 Prüfer 序列后原树中会剩下两个结点，其中一个一定是编号最大的点 n 。
2. 每个结点在序列中出现的次数是其度数减 1。（没有出现的就是叶结点）

用 Prüfer 序列重建树:

重建树的方法是类似的。根据 Prüfer 序列的性质，我们可以得到原树上每个点的度数。然后你也可以得到编号最小的叶结点，而这个结点一定与 Prüfer 序列的第一个数连接。然后我们同时删掉这两个结点的度数。

讲到这里也许你已经知道该怎么做了。每次我们选择一个度数为 1 的最小的结点编号，与当前枚举到的 Prüfer 序列的点连接，然后同时减掉两个点的度。到最后我们剩下两个度数为 1 的点，其中一个是结点 n 。就把它们建立连接。使用堆维护这个过程，在减度数的过程中如果发现度数减到 1 就把这个结点添加到堆中，这样做的复杂度是 $O(n \log n)$ 的。

```

1 // 原文摘代码
2 vector<pair<int, int>> pruefer_decode(vector<int> const& code) {

```

```

3   int n = code.size() + 2;
4   vector<int> degree(n, 1);
5   for (int i : code) degree[i]++;
6
7   set<int> leaves;
8   for (int i = 0; i < n; i++)
9       if (degree[i] == 1) leaves.insert(i);
10
11  vector<pair<int, int>> edges;
12  for (int v : code) {
13      int leaf = *leaves.begin();
14      leaves.erase(leaves.begin());
15
16      edges.emplace_back(leaf, v);
17      if (--degree[v] == 1) leaves.insert(v);
18  }
19  edges.emplace_back(*leaves.begin(), n - 1);
20  return edges;
21  }

```

线性时间重建树

同线性构造 Prüfer 序列的方法。在删度数的时候会产生新的叶结点，于是判断这个叶结点与指针 p 的大小关系，如果更小就优先考虑它。

```

1   // 原文摘代码
2   vector<pair<int, int>> pruefer_decode(vector<int> const& code) {
3       int n = code.size() + 2;
4       vector<int> degree(n, 1);
5       for (int i : code) degree[i]++;
6
7       int ptr = 0;
8       while (degree[ptr] != 1) ptr++;
9       int leaf = ptr;
10
11  vector<pair<int, int>> edges;
12  for (int v : code) {
13      edges.emplace_back(leaf, v);
14      if (--degree[v] == 1 && v < ptr) {
15          leaf = v;
16      } else {
17          ptr++;
18          while (degree[ptr] != 1) ptr++;
19          leaf = ptr;
20      }
21  }
22  edges.emplace_back(leaf, n - 1);
23  return edges;
24  }

```

Cayley 公式 (Cayley's formula):

完全图 K_n 有 n^{n-2} 棵生成树。

图连通方案数:

Prüfer 序列可能比你想得还强大。它能创造比 Cayley 公式更通用的公式。比如以下问题:

一个 n 个点 m 条边的带标号无向图有 k 个连通块。我们希望添加 $k - 1$ 条边使得整个图连通。求方案数。

设 s_i 表示每个连通块的数量。我们对 k 个连通块构造 Prüfer 序列，然后你发现这并不是普通的 Prüfer 序列。因为每个连通块的连接方法很多。不能直接淦就设啊。于是设 d_i 为第 i 个连通块的度数。由于度数之和是边数的两倍，于是

$$\sum_{i=1}^k d_i = 2k - 2$$

则对于给定的 d 序列构造 Prüfer 序列的方案数是

$$\binom{k-2}{d_1-1, d_2-1, \dots, d_k-1} = \frac{(k-2)!}{(d_1-1)!(d_2-1)! \cdots (d_k-1)!}$$

对于第 i 个连通块，它的连接方式有 $s_i^{d_i}$ 种，因此对于给定 d 序列使图连通的方案数是

$$\binom{k-2}{d_1-1, d_2-1, \dots, d_k-1} \cdot \prod_{i=1}^k s_i^{d_i}$$

现在我们要枚举 d 序列，式子变成

$$\sum_{\substack{d_i \geq 1 \\ \sum_{i=1}^k d_i = 2k-2}} \binom{k-2}{d_1-1, d_2-1, \dots, d_k-1} \cdot \prod_{i=1}^k s_i^{d_i}$$

好的这是一个非常不喜闻乐见的式子。但是别慌！我们有多元二项式定理：

$$(x_1 + \dots + x_m)^p = \sum_{\substack{c_i \geq 0 \\ \sum_{i=1}^m c_i = p}} \binom{p}{c_1, c_2, \dots, c_m} \cdot \prod_{i=1}^m x_i^{c_i}$$

那么我们对原式做一下换元，设 $e_i = d_i - 1$ ，显然

$$\sum_{i=1}^k e_i = k - 2$$

于是原式变成

$$\sum_{\substack{e_i \geq 0 \\ \sum_{i=1}^k e_i = k-2}} \binom{k-2}{e_1, e_2, \dots, e_k} \cdot \prod_{i=1}^k s_i^{e_i+1}$$

化简得到

$$(s_1 + s_2 + \dots + s_k)^{k-2} \cdot \prod_{i=1}^k s_i$$

即

$$n^{k-2} \cdot \prod_{i=1}^k s_i$$

这就是答案啦。